

Labolatorium 12

May 28, 2026

Jakub Orchowski, s223281

1 CEL ĆWICZENIA

Zapoznanie się z teorią i praktycznym zastosowaniem dyskretnej transformaty Fouriera (DFT) oraz szybkiej transformaty Fouriera (FFT) w analizie sygnałów 1D i 2D, w tym filtracji w dziedzinie częstotliwości.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from mpl_toolkits.mplot3d import Axes3D
from IPython.display import Audio, display
import wave
%matplotlib inline
```

2 Zadania

2.1 Zadanie 1

Dany jest sygnał dyskretny o równaniu: $x(n) = \cos(0.314n + 2.5) + 2 \cos(1.57n - 0.18)$.

- Wygeneruj sekwencję złożoną z 32 próbek tego sygnału (dla $n = 0, \dots, 31$). Oblicz transformatę Fouriera tej sekwencji korzystając z ogólnego wzoru na transformatę Fouriera sygnałów dyskretnych, a następnie oblicz jej dyskretną/szybką transformatę Fouriera (FFT). Wyświetl i porównaj amplitudy obu transformat.
- Wygeneruj sekwencję złożoną z 1024 próbek tego sygnału (dla $n = 0, \dots, 1023$). Oblicz szybką transformatę Fouriera (FFT) tej sekwencji. Wyświetl amplitudę tej transformaty i porównaj wynik z punktem (a).

Dla lepszego zrozumienia wyników, w obu przypadkach wyświetl transformaty Fouriera zarówno w wersji oryginalnej, jak i zmodyfikowanej przez funkcję `fftshift`.

```
[2]: def x_signal(n):
    """Sygnał dyskretny x(n)."""
    return np.cos(0.314 * n + 2.5) + 2 * np.cos(1.57 * n - 0.18)

def dft_manual(x):
```

```

"""Dyskretna transformata Fouriera zdefiniowana wzorem ogólnym."""
N = len(x)
n = np.arange(N)
k = n.reshape((N, 1))
W = np.exp(-2j * np.pi * k * n / N)
return (W @ x).astype(np.complex128)

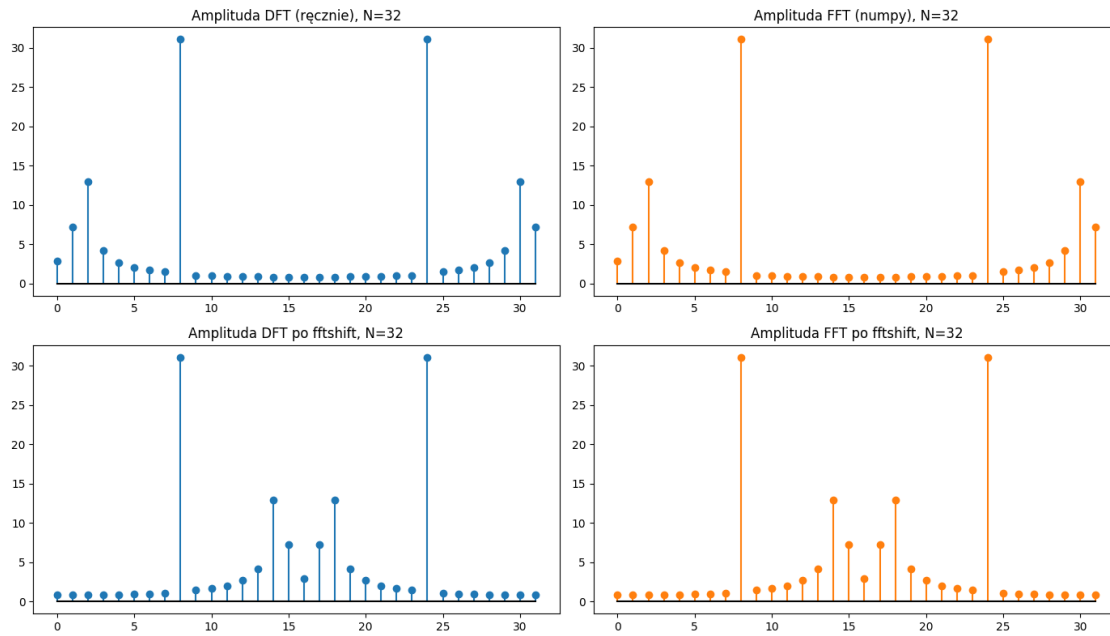
# a) N = 32
N32 = 32
n32 = np.arange(N32)
x32 = x_signal(n32)

dft32 = dft_manual(x32)
fft32 = np.fft.fft(x32)

fig, axes = plt.subplots(2, 2, figsize=(14, 8))
axes[0, 0].stem(np.abs(dft32), linefmt='C0-', markerfmt='C0o', basefmt='k-')
axes[0, 0].set_title('Amplituda DFT (ręcznie), N=32')
axes[0, 1].stem(np.abs(fft32), linefmt='C1-', markerfmt='C1o', basefmt='k-')
axes[0, 1].set_title('Amplituda FFT (numpy), N=32')
axes[1, 0].stem(np.abs(np.fft.fftshift(dft32)), linefmt='C0-', markerfmt='C0o',
    ↪basefmt='k-')
axes[1, 0].set_title('Amplituda DFT po fftshift, N=32')
axes[1, 1].stem(np.abs(np.fft.fftshift(fft32)), linefmt='C1-', markerfmt='C1o',
    ↪basefmt='k-')
axes[1, 1].set_title('Amplituda FFT po fftshift, N=32')
plt.tight_layout()
plt.show()

print(f'Maksymalna różnica |DFT - FFT| dla N=32: {np.max(np.abs(dft32 - fft32)):
    ↪.2e}')

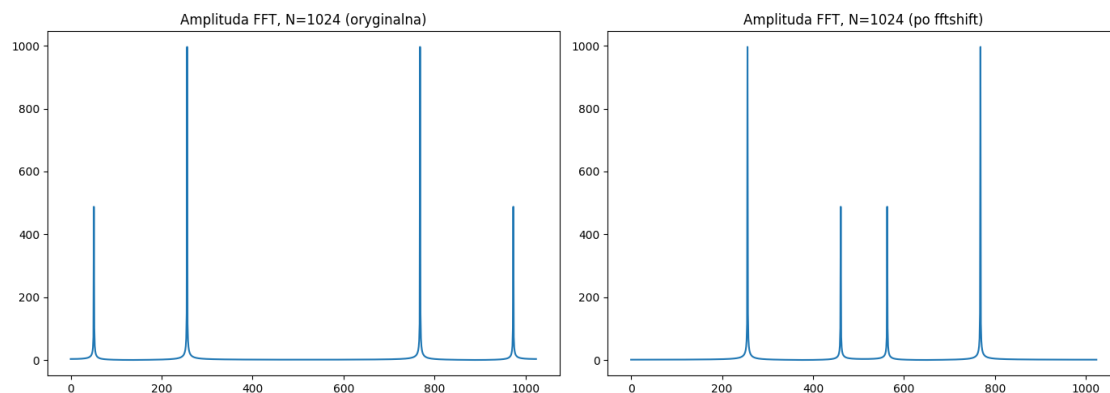
```



Maksymalna różnica $|DFT - FFT|$ dla $N=32$: $2.07e-13$

```
[3]: # b) N = 1024
N1024 = 1024
n1024 = np.arange(N1024)
x1024 = x_signal(n1024)
fft1024 = np.fft.fft(x1024)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(np.abs(fft1024))
axes[0].set_title('Amplituda FFT, N=1024 (oryginalna)')
axes[1].plot(np.abs(np.fft.fftshift(fft1024)))
axes[1].set_title('Amplituda FFT, N=1024 (po fftshift)')
plt.tight_layout()
plt.show()
```



2.1.1 Wnioski

Dla $N=32$ ręcznie obliczona DFT i biblioteczna FFT dają identyczne wyniki (różnica na poziomie błędu numerycznego), co potwierdza poprawność implementacji. Zwiększenie liczby próbek do $N=1024$ znacząco poprawia rozdzielczość widma — widoczne są wyraźne piki odpowiadające składowym cosinusoidalnym sygnałom. Funkcja `fftshift` przesuwą składową zerowej częstotliwości do środka, ułatwiając interpretację widma.

2.2 Zadanie 2

Dany jest zdyskretyzowany sygnał akustyczny `icing2.wav` (częstotliwość próbkowania 44100 Hz).

- Oblicz szybką transformatę Fouriera (FFT) tego sygnału, a następnie pomiń (wyzeruj) składowe tej transformaty o częstotliwościach większych niż 2205 Hz. Na podstawie tak zmodyfikowanej transformaty odtwórz oryginalny sygnał. Wyświetl amplitudę obliczonej transformaty FFT przed wyzerowaniem wybranych składowych oraz po ich wyzerowaniu.
- W transformacie obliczonej w (a) pomiń (wyzeruj) jej składowe o częstotliwościach mniejszych niż 1103 Hz. Na podstawie tak zmodyfikowanej transformaty odtwórz oryginalny sygnał. Wyświetl amplitudę obliczonej transformaty FFT przed wyzerowaniem wybranych składowych oraz po ich wyzerowaniu.

Dla lepszej wizualizacji wyników, w każdym przypadku wyświetl przede wszystkim transformatę zmodyfikowaną przez funkcję `fftshift`.

```
[4]: with wave.open('icing2.wav', 'rb') as wf:
    fs = wf.getframerate()
    n_frames = wf.getnframes()
    n_channels = wf.getnchannels()
    raw = wf.readframes(n_frames)

    audio = np.frombuffer(raw, dtype=np.int16)
    if n_channels > 1:
        audio = audio.reshape(-1, n_channels)[: , 0]
    audio = audio.astype(np.float64)

    N = len(audio)
    fft_audio = np.fft.fft(audio)
    freqs = np.fft.fftfreq(N, d=1/fs)

    # a) Filtr dolnoprzepustowy |f| <= 2205 Hz
    mask_a = np.abs(freqs) <= 2205
    fft_a = fft_audio.copy()
    fft_a[~mask_a] = 0
    audio_a = np.fft.ifft(fft_a).real

    fig, axes = plt.subplots(2, 2, figsize=(14, 8))
```

```

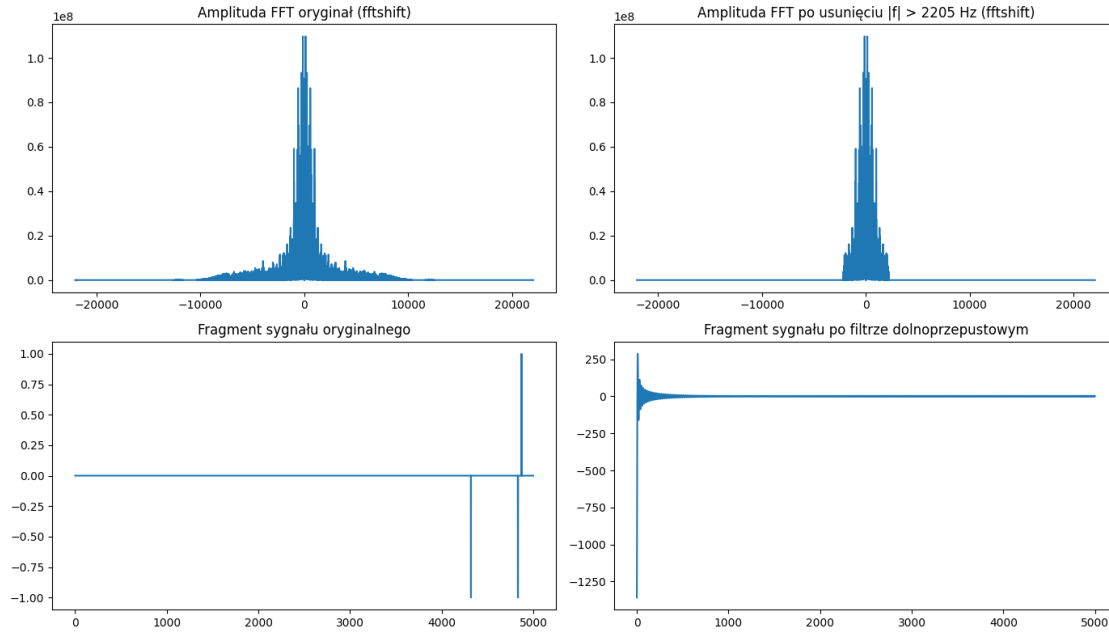
axes[0, 0].plot(np.fft.fftshift(freqs), np.fft.fftshift(np.abs(fft_audio)))
axes[0, 0].set_title('Amplituda FFT oryginal (fftshift)')
axes[0, 1].plot(np.fft.fftshift(freqs), np.fft.fftshift(np.abs(fft_a)))
axes[0, 1].set_title('Amplituda FFT po usunięciu  $|f| > 2205$  Hz (fftshift)')
axes[1, 0].plot(audio[:5000])
axes[1, 0].set_title('Fragment sygnału oryginalnego')
axes[1, 1].plot(audio_a[:5000])
axes[1, 1].set_title('Fragment sygnału po filtrze dolnoprzepustowym')
plt.tight_layout()
plt.show()

def save_wav(data, filename, sample_rate):
    """Zapisuje sygnał do pliku WAV (16-bit)."""
    data = np.clip(data, -32768, 32767).astype(np.int16)
    with wave.open(filename, 'wb') as wf:
        wf.setnchannels(1)
        wf.setsampwidth(2)
        wf.setframerate(sample_rate)
        wf.writeframes(data.tobytes())

save_wav(audio_a, 'icing2_lowpass.wav', fs)
print('Zapisano: icing2_lowpass.wav')
def save_wav(data, filename, sample_rate):
    """Zapisuje sygnał do pliku WAV (16-bit)."""
    data = np.clip(data, -32768, 32767).astype(np.int16)
    with wave.open(filename, 'wb') as wf:
        wf.setnchannels(1)
        wf.setsampwidth(2)
        wf.setframerate(sample_rate)
        wf.writeframes(data.tobytes())

save_wav(audio_a, 'icing2_lowpass.wav', fs)
print('Zapisano: icing2_lowpass.wav')
print('Odtwarzanie sygnału po filtracji dolnoprzepustowej ( $|f| \leq 2205$  Hz):')
display(Audio(audio_a, rate=fs))

```



Zapisano: icing2_lowpass.wav

Zapisano: icing2_lowpass.wav

Odtwarzanie sygnału po filtracji dolnoprzepustowej ($|f| \leq 2205$ Hz):

<IPython.lib.display.Audio object>

```
[5]: # b) Filtr pasmowoprzepustowy 1103 Hz <= |f| <= 2205 Hz
mask_b = (np.abs(freqs) >= 1103) & (np.abs(freqs) <= 2205)
fft_b = fft_audio.copy()
fft_b[~mask_b] = 0
audio_b = np.fft.ifft(fft_b).real

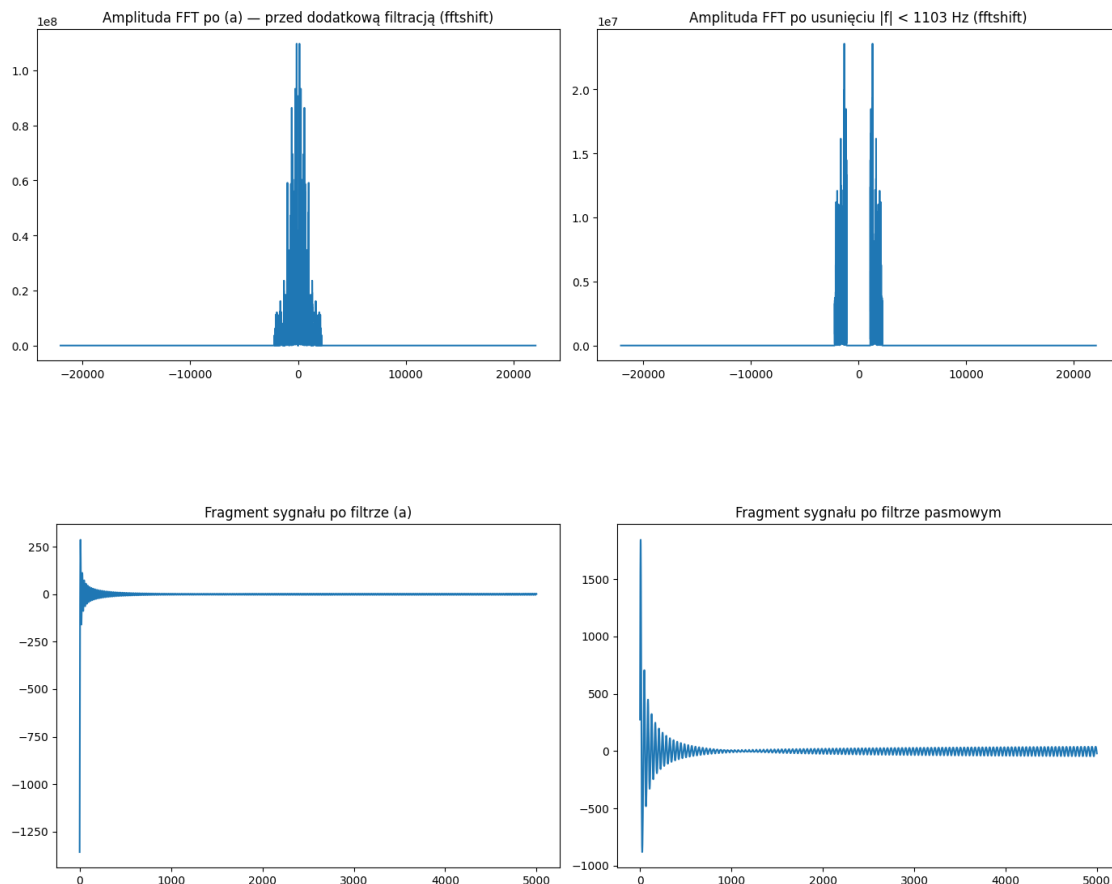
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(np.fft.fftshift(freqs), np.fft.fftshift(np.abs(fft_a)))
axes[0].set_title('Amplituda FFT po (a) - przed dodatkową filtracją (fftshift)')
axes[1].plot(np.fft.fftshift(freqs), np.fft.fftshift(np.abs(fft_b)))
axes[1].set_title('Amplituda FFT po usunięciu |f| < 1103 Hz (fftshift)')
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(audio_a[:5000])
axes[0].set_title('Fragment sygnału po filtrze (a)')
axes[1].plot(audio_b[:5000])
axes[1].set_title('Fragment sygnału po filtrze pasmowym')
plt.tight_layout()
plt.show()
```

```

save_wav(audio_b, 'icing2_bandpass.wav', fs)
print('Zapisano: icing2_bandpass.wav')
save_wav(audio_b, 'icing2_bandpass.wav', fs)
print('Zapisano: icing2_bandpass.wav')
print('Odtwarzanie sygnału po filtracji pasmowoprzepustowej (1103-2205 Hz):')
display(Audio(audio_b, rate=fs))

```



```

Zapisano: icing2_bandpass.wav
Zapisano: icing2_bandpass.wav
Odtwarzanie sygnału po filtracji pasmowoprzepustowej (1103-2205 Hz):
<IPython.lib.display.Audio object>

```

2.2.1 Wnioski

Filtr dolnoprzepustowy usuwający składowe o częstotliwościach powyżej 2205 Hz znacząco tłumi wysokie tony w sygnale audio, co słyszalne jako przyciemnienie brzmienia. Dodatkowe zastosowanie filtru górnoprzepustowego (usunięcie częstotliwości poniżej 1103 Hz) pozostawia wąskie pasmo średnich częstotliwości, co daje efekt „telefonicznego” brzmienia sygnału. Filtracja w dziedzinie częstotliwości jest efektywnym narzędziem do modyfikacji sygnałów akustycznych.

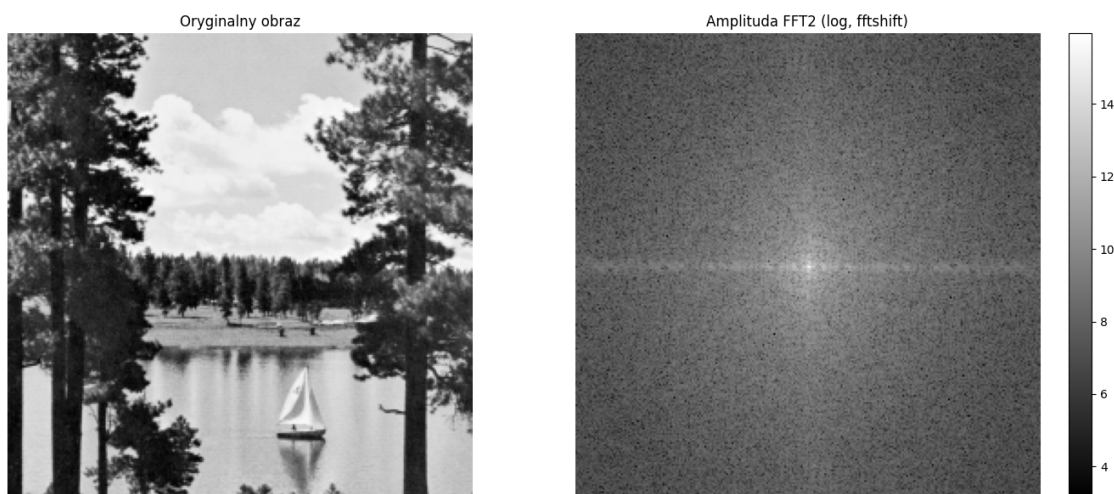
2.3 Zadanie 3

Oblicz 2-wymiarową szybką transformatę Fouriera (FFT) załączonego obrazka LAKE.bmp. Wyświetl amplitudę tej transformaty (używając na przykład funkcji `mesh`). Dla lepszego zrozumienia wyników, wyświetl również transformatę zmodyfikowaną przez funkcję `fftshift`.

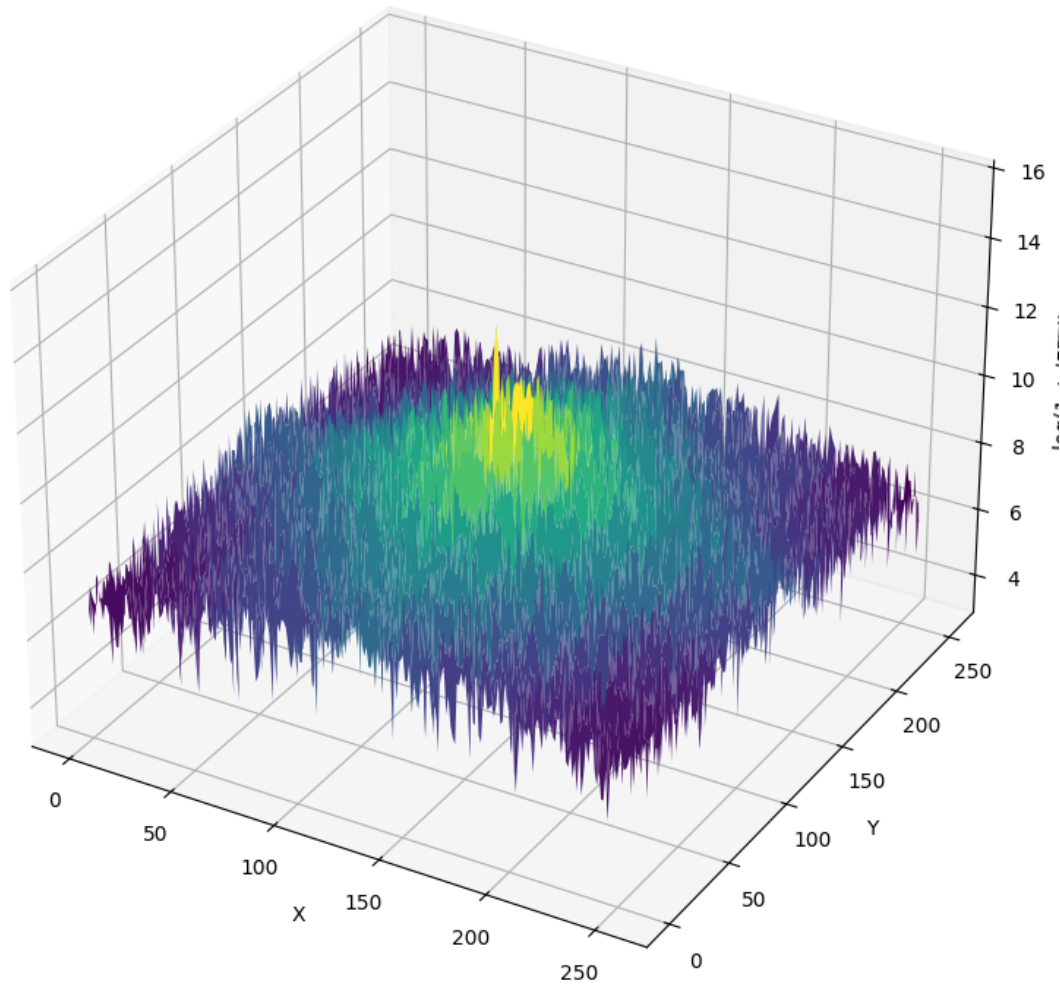
```
[6]: img = np.array(Image.open('LAKE.bmp').convert('L'))
fft2_img = np.fft.fft2(img)
fft2_shifted = np.fft.fftshift(fft2_img)
amplitude = np.log(1 + np.abs(fft2_shifted))

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
axes[0].imshow(img, cmap='gray')
axes[0].set_title('Oryginalny obraz')
axes[0].axis('off')
im = axes[1].imshow(amplitude, cmap='gray')
axes[1].set_title('Amplituda FFT2 (log, fftshift)')
axes[1].axis('off')
plt.colorbar(im, ax=axes[1], fraction=0.046)
plt.tight_layout()
plt.show()

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
X, Y = np.meshgrid(np.arange(img.shape[1]), np.arange(img.shape[0]))
ax.plot_surface(X, Y, amplitude, cmap='viridis', rstride=20, cstride=20,
               antialiased=True)
ax.set_title('Amplituda FFT2 - wizualizacja 3D (log, fftshift)')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('log(1 + |FFT|)')
plt.tight_layout()
plt.show()
```



Amplituda FFT2 — wizualizacja 3D (log, fftshift)



2.3.1 Wnioski

Widmo amplitudowe 2D FFT obrazu `LAKE.bmp` wykazuje silną składową zerowej częstotliwości (jasny punkt centralny po `fftshift`), co odpowiada średniej jasności obrazu. Energia widma koncentruje się wokół środka, co świadczy o dominacji niskich częstotliwości przestrzennych. Wizualizacja 3D pozwala zaobserwować charakterystyczne „piki” odpowiadające powtarzającym się wzorcom w obrazie.

2.4 Zadanie 4

- a) W transformatie Fouriera z Zadania 3 pomini (zastąp zerami) te składowe transformaty, w których częstotliwość w kierunku X (ω_x) jest większa niż $\pi/12$. Na podstawie tak zmodyfikowanej transformaty odtwórz obraz pierwotny stosując odwrotną transformatę Fouriera. Wyświetl również zmodyfikowaną transformatę (zalecana modyfikacja przez funkcję `fftshift`).

Wykonaj podobne operacje, ale:

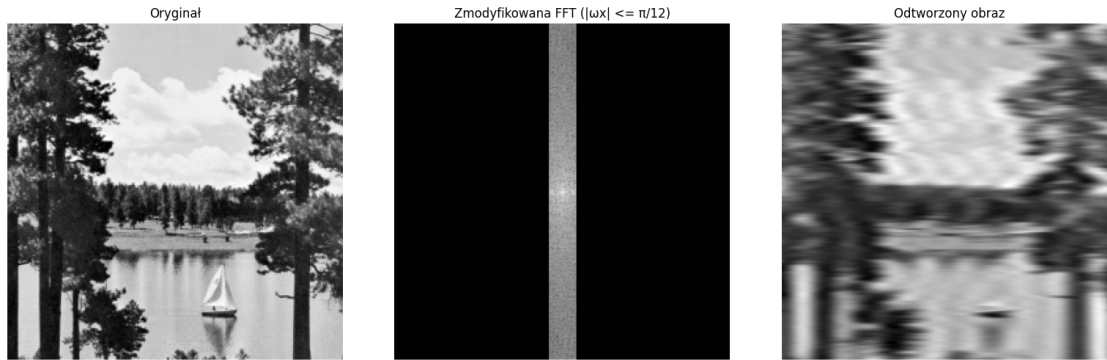
- b) Pomiń te składowe transformaty, w których częstotliwość w kierunku Y (ω_y) jest mniejsza niż $\pi/12$.
- c) Pomiń te składowe transformaty, w których albo częstotliwość w kierunku X (ω_x), albo częstotliwość w kierunku Y (ω_y) jest większa niż $\pi/10$.

W obu przypadkach wyświetl odtworzony obraz i porównaj z obrazem oryginalnym. Wyświetl również otrzymane transformaty (najlepiej po modyfikacji przez funkcję `fftshift`).

```
[7]: H, W = img.shape
freqs_y = np.fft.fftshift(np.fft.fftfreq(H, d=1)) * 2 * np.pi
freqs_x = np.fft.fftshift(np.fft.fftfreq(W, d=1)) * 2 * np.pi
omega_x, omega_y = np.meshgrid(freqs_x, freqs_y)

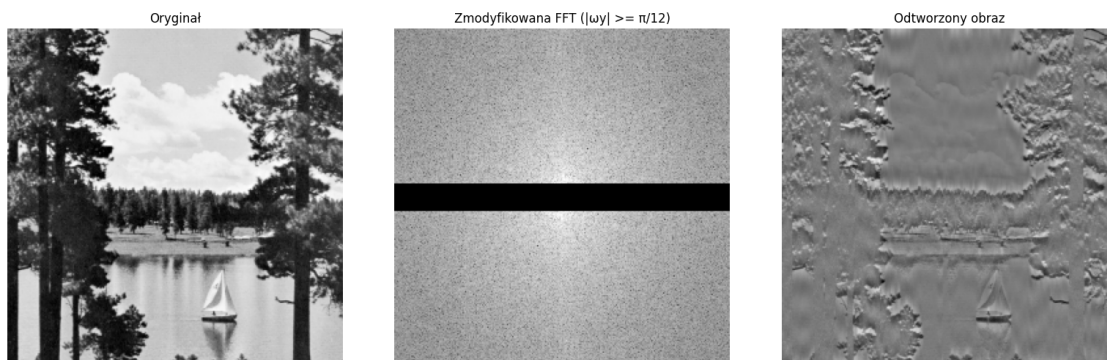
# a) Filtr dolnoprzepustowy w kierunku X: |x| <= /12
mask_a = np.abs(omega_x) <= np.pi / 12
fft4a = fft2_shifted * mask_a
img4a = np.fft.ifft2(np.fft.ifftshift(fft4a)).real

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
axes[0].imshow(img, cmap='gray')
axes[0].set_title('Oryginał')
axes[0].axis('off')
axes[1].imshow(np.log(1 + np.abs(fft4a)), cmap='gray')
axes[1].set_title('Zmodyfikowana FFT (|x| <= /12)')
axes[1].axis('off')
axes[2].imshow(img4a, cmap='gray')
axes[2].set_title('Odtworzony obraz')
axes[2].axis('off')
plt.tight_layout()
plt.show()
```



```
[8]: # b) Filtr górnoprzepustowy w kierunku Y: |y| ≥ /12
mask_b = np.abs(omega_y) >= np.pi / 12
fft4b = fft2_shifted * mask_b
img4b = np.fft.ifft2(np.fft.ifftshift(fft4b)).real

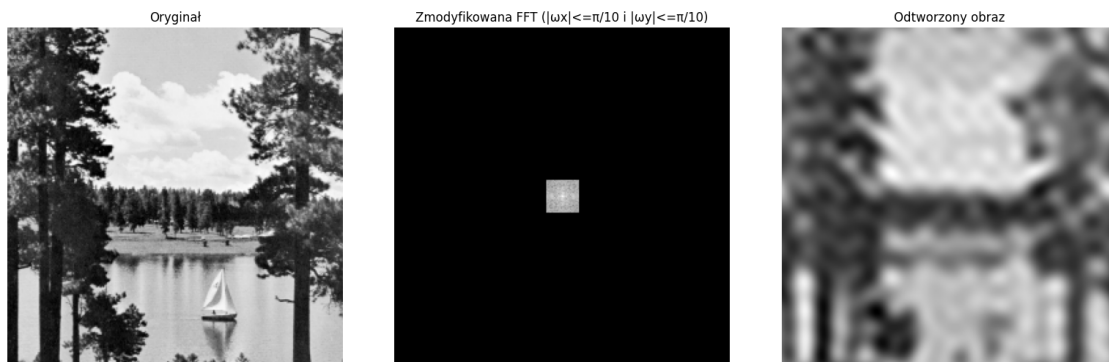
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
axes[0].imshow(img, cmap='gray')
axes[0].set_title('Oryginał')
axes[0].axis('off')
axes[1].imshow(np.log(1 + np.abs(fft4b)), cmap='gray')
axes[1].set_title('Zmodyfikowana FFT (|y| ≥ /12)')
axes[1].axis('off')
axes[2].imshow(img4b, cmap='gray')
axes[2].set_title('Odtworzony obraz')
axes[2].axis('off')
plt.tight_layout()
plt.show()
```



```
[9]: # c) Filtr dolnoprzepustowy 2D: |x| ≤ /10 i |y| ≤ /10
mask_c = (np.abs(omega_x) <= np.pi / 10) & (np.abs(omega_y) <= np.pi / 10)
```

```
fft4c = fft2_shifted * mask_c
img4c = np.fft.ifft2(np.fft.ifftshift(fft4c)).real

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
axes[0].imshow(img, cmap='gray')
axes[0].set_title('Oryginał')
axes[0].axis('off')
axes[1].imshow(np.log(1 + np.abs(fft4c)), cmap='gray')
axes[1].set_title('Zmodyfikowana FFT (|x|<= /10 i |y|<= /10)')
axes[1].axis('off')
axes[2].imshow(img4c, cmap='gray')
axes[2].set_title('Odtworzony obraz')
axes[2].axis('off')
plt.tight_layout()
plt.show()
```



2.4.1 Wnioski

Usunięcie wysokich częstotliwości w kierunku X (a) powoduje rozmycie obrazu w poziomie, natomiast zachowanie pionowych krawędzi. Filtr górnoprzepustowy w kierunku Y (b) podkreśla poziome kontury i usuwa powolne zmiany jasności w pionie. Filtr dolnoprzepustowy 2D (c) znacząco rozmyca cały obraz, zachowując tylko najgrubsze struktury — co ilustruje, że niskie częstotliwości przestrzenne niosą informację o ogólnej kompozycji obrazu, a wysokie o drobnych szczegółach.